Biweekly Assessment BW03 — Weeks 5–6

Sessions covered: S21–30
Topics: Tuples, sets, dicts, structure choice, stacks, functions intro (def/return/params), scope, lambda, `map` / `filter` / `reduce`

Timebox: 45–60 minutes (homework or Friday of the even week).
Rules: Closed notes unless your trainer allows open notes. No copying.
Coding constraint: Dicts/sets OK. Simple `def`, `lambda`, `map` / `filter` / `reduce` OK. No full decorator/generator write-ups yet (Week 7).

A. Multiple Choice (12)

Circle the best option.

1. Tuples are:
   a) Mutable
   b) Immutable
   c) Always empty
   d) Same as sets
2. Sets store:
   a) Unique elements
   b) Only strings
   c) Ordered always (guarantee)
   d) Duplicates kept
3. `dict.fromkeys` is useful for:
   a) Opening files
   b) Installing pip
   c) Sorting ints
   d) Order-preserving unique keys
4. Dict lookup uses:
   a) SQL
   b) Indexes only
   c) Keys
   d) Only loops
5. `contacts.get("Ravi")` if missing:
   a) KeyError
   b) None (default)
   c) 0
   d) False always
6. A stack is typically:
   a) LIFO
   b) Random
   c) Sorted tree
   d) FIFO
7. `list.pop()` without index removes:
   a) Middle item
   b) Nothing

c) First item
d) Last item

8. A function starts with:
   a) method
   b) func
   c) def
   d) function
9. Function with no `return` returns:
   a) 0
   b) None
   c) Empty string
   d) Error always
10. Parameter vs argument:
    a) Parameter in def; argument in call
    b) Argument in def only
    c) Neither used in Python
    d) Same word always
11. Best structure for roll_no → name:
    a) set
    b) dict
    c) list
    d) tuple
12. `list(filter(lambda m: m >= 75, [60, 80, 90]))` returns:
    a) `[60, 80, 90]`
    b) `[80, 90]`
    c) `True`
    d) `230`

B. Coding (2)

1. Build a contact-book **menu** with a dict (add / lookup / delete / list / quit). Missing lookup must not crash. Script style or simple functions OK.

```
# your code
```

2. Write `def deposit(balance, amount)` and `def withdraw(balance, amount)` that print a message and return unchanged balance on bad amount/overdraft; otherwise return new balance.

```
# your code
```

C. Short answer (1)

Why wrap **SecureBank** ATM logic in functions instead of one giant script? Two concrete benefits.

(Write 3–6 sentences.)